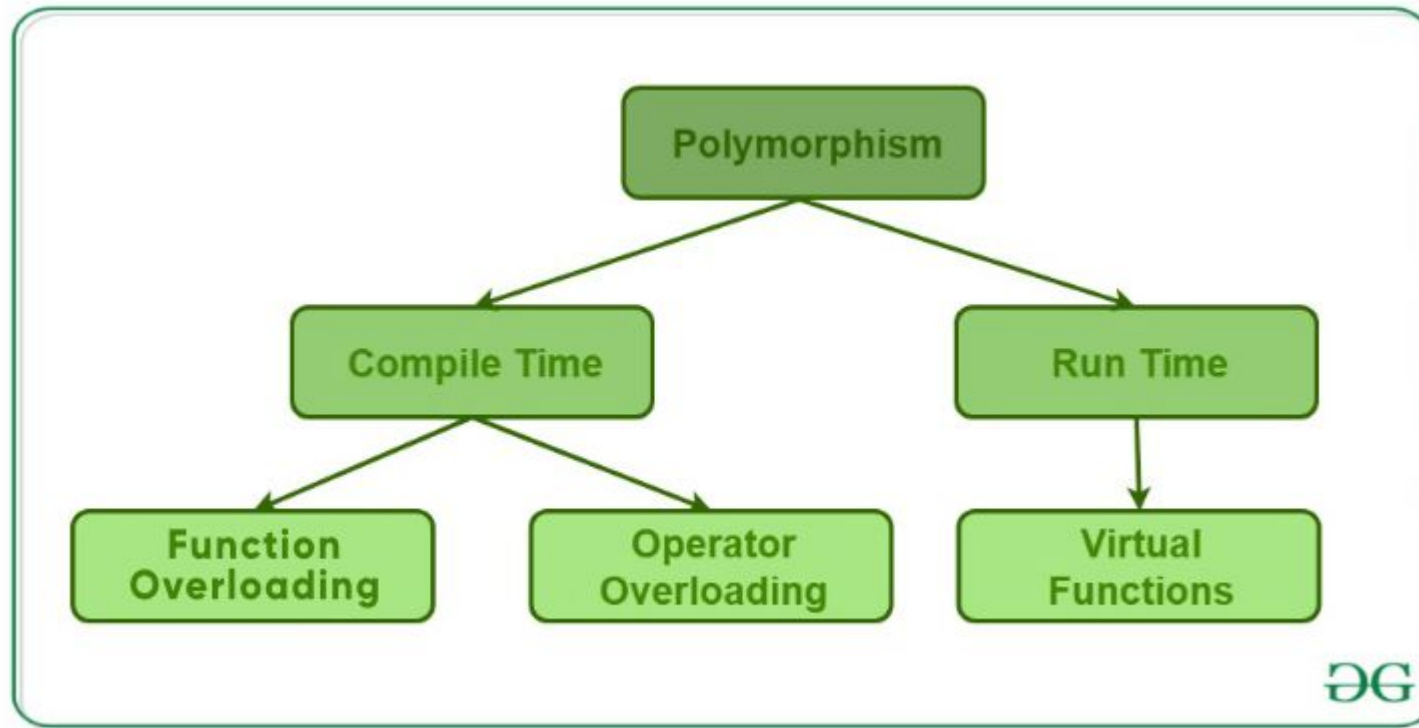


Polymorphism in C++

- The word polymorphism means having many forms.
- In simple words, we can define polymorphism as the ability of a message to be displayed in more than one form.
- Polymorphism is considered as one of the important features of Object Oriented Programming.
- Example: name/nick name

- In C++ polymorphism is mainly divided into two types:
- Compile time Polymorphism
- Runtime Polymorphism



Compile time polymorphism: This type of polymorphism is achieved by function overloading or operator overloading.

Function Overloading

- C++ allows you to specify more than one definition for a **function** name or an **operator** in the same scope, which is called **function overloading** and **operator overloading** respectively.
- An overloaded declaration is a declaration that is declared with the same name as a previously declared declaration in the same scope, except that both declarations have different arguments and obviously different definition (implementation).
- When you call an overloaded **function** or **operator**, the compiler determines the most appropriate definition to use, by comparing the argument types you have used to call the function or operator with the parameter types specified in the definitions. The process of selecting the most appropriate overloaded function or operator is called **overload resolution**.

```
#include <iostream>
using namespace std;
class printData
{ public:
    void print(int i)
    {    cout << "Printing int: " << i << endl;    }
    void print(double f)
    {    cout << "Printing float: " << f << endl;    }
    void print(char* c) {    cout << "Printing character: " << c << endl;    }
};
int main(void)
{ printData pd; // Call print to print integer
  pd.print(5); // Call print to print float
  pd.print(500.263); // Call print to print character
  pd.print("Hello C++");
  return 0;
}
```

```
Printing int: 5
Printing float: 500.263
Printing character: Hello C++
```

Functions that cannot be overloaded in C++

In C++, following function declarations **cannot** be overloaded.

- 1) Function declarations that differ only in the return type. For example, the following program fails in compilation.

```
#include<iostream>
int foo() {
    return 10;
}

char foo() {
    return 'a';
}

int main()
{
    char x = foo();
    getchar();
    return 0;
}
```

2) Member function declarations with the same name and the same parameter-type-list cannot be overloaded if any of them is a static member function declaration. For example, following program fails in compilation

```
#include<iostream>
class Test {
static void fun(int i) {}
void fun(int i) {}
};

int main()
{
Test t;
getchar();
return 0;
}
```

3) Parameter declarations that differ only in a pointer * versus an array [] are equivalent. That is, the array declaration is adjusted to become a pointer declaration. Only the second and subsequent array dimensions are significant in parameter types. For example, following two function declarations are equivalent

```
int fun(int *ptr);  
int fun(int ptr[]); // redeclaration of fun(int *ptr)
```


4) Parameter declarations that differ only in the presence or absence of const. That is, the const parameter type are ignored when determining which function is being declared, defined, or called. For example, following program fails in compilation with error *“redefinition of `int f(int)’* “

```
#include<iostream>
#include<stdio.h>

using namespace std;

int f ( int x) {
    return x+10;
}

int f ( const int x) {
    return x+10;
}

int main() {
    getchar();
    return 0;
}
```

5) Two parameter declarations that differ only in their default arguments are equivalent. For example, following program fails in compilation with error *“redefinition of ‘int f(int, int)’ “*

```
#include<iostream>
```

```
#include<stdio.h>
```

```
using namespace std;
```

```
int f ( int x, int y) {  
    return x+10;  
}
```

```
int f ( int x, int y = 10) {  
    return x+y;  
}
```

```
int main() {  
    getchar();  
    return 0;  
}
```